Wenn wir beispielsweise einen intelligenten Zuteilungsalgorithmus implementieren, der ein Fahrzeug für einen Taxiruf auswählt, oder wenn wir ein gutes Schema für die Verteilung der wartenden Fahrzeuge entwickeln, dann könnten wir entscheiden, dass diese Algorithmen auch im produktiven Betrieb eingesetzt werden können. Die visuelle Repräsentation der Positionen der Fahrzeuge könnte ebenfalls helfen.

Mit anderen Worten: Es besteht Potenzial dafür, dass aus der Simulation ein Verwaltungssystem für den Betrieb eines echten Taxiunternehmens wird. Die Struktur der Anwendung würde sich dabei natürlich ändern: Das Programm würde die Taxis nicht mehr kontrollieren und bewegen, sondern lediglich ihre Position verfolgen, die durch Empfänger für GPS (das Global Positioning System) in allen Fahrzeugen geliefert wird. Viele der Klassen aus der Simulation könnten jedoch ohne große Änderungen wiederverwendet werden. Dies demonstriert die Stärke einer guten Klassenstruktur und eines guten Entwurfs.

16.5 Ein weiteres Beispiel

Es gibt viele weitere Projekte, die Sie ganz ähnlich wie das *Taxi*-Projekt durchführen können. Ein beliebtes Beispiel ist die Verwaltung von Fahrstühlen in einem großen Gebäude. Hier wird die Koordination zwischen den einzelnen Fahrstühlen besonders wichtig. In einem geschlossenen Gebäude kann es außerdem möglich sein, die Anzahl der Personen pro Stockwerk zu schätzen und damit den Bedarf vorherzusehen. Außerdem kann zeitabhängiges Verhalten berücksichtigt werden – morgendliches Eintreffen, abendliches Verlassen des Gebäudes und lokale Spitzen um die Mittagszeit herum.

Benutzen Sie den Ansatz, den wir in diesem Kapitel dargestellt haben, um die Simulation eines Gebäudes mit mehreren Fahrstühlen zu implementieren.

16.6 Ein Blick nach vorn

Wir können Sie nur bis zu einem bestimmten Punkt bringen, indem wir unsere eigenen Ideen für Projekte präsentieren und zeigen, wie wir sie entwickeln würden. Sie werden feststellen, dass Sie sehr viel weiter gehen können, wenn Sie Ihre eigenen Ideen für Projekte entwickeln und diese auf Ihre eigene Weise implementieren. Wählen Sie ein Thema, das Sie interessiert, und durchlaufen Sie die Stufen, die wir umrissen haben: Analysieren Sie das Problem, arbeiten Sie Szenarios aus, machen Sie einen ersten Entwurf, planen Sie einige Implementierungsstufen und fangen Sie an.

Der Entwurf und die Implementierung von Programmen ist eine spannende und kreative Tätigkeit. Wie bei jeder lohnenswerten Tätigkeit braucht es einige Zeit, ehe man sie beherrscht. Lassen Sie sich also nicht entmutigen, wenn Ihre ersten Versuche sehr lange brauchen oder voller Fehler sind. Das ist völlig normal; Sie werden sich mit wachsender Erfahrung verbessern. Seien Sie zu Anfang nicht zu ehrgeizig und rechnen Sie damit, dass Sie Ihre Vorstellungen unterwegs anpassen müssen – all das gehört zum natürlichen Lernprozess.

Vor allem aber gilt: Haben Sie Spaß an der Sache!

Lizenziert für Hochschule Luzern, 147.88.200.39 am 22.02.2023 um 10:51 Uhr

ANHANG

A

Arbeiten mit BlueJ-Projekten

A.1 BlueJ installieren

Um mit BlueJ arbeiten zu können, müssen Sie das BlueJ-System herunterladen und installieren. Das System können Sie kostenlos von *http://www.bluej.org/* herunterladen. Der Download für Windows und Mac OS enthält eine Kopie des JDK (das Java-System) – es muss nicht separat installiert werden. Unter Linux werden die Paketabhängigkeiten automatisch die richtige JDK-Version installieren, falls nötig.

A.2 Ein Projekt öffnen

Sie können alle Beispielprojekte von der Companion Website dieses Buch herunterladen:

http://www.pearson-studium.de/java-lernen-mit-bluej.html

Die englische Version der Quelltexte finden Sie unter:

http://www.bluej.org/objects-first/

Laden Sie die zip-Datei herunter und entpacken Sie diese im Ordner *Projekte*. Nachdem Sie BlueJ installiert und mit einem Doppelklick auf das sein Symbol gestartet haben, wählen Sie den Eintrag PROJEKT ÖFFNEN… im Menü PROJEKT. Navigieren Sie in den Ordner *Projekte* und wählen Sie eines der Projekte. (Sie können mehrere Projekte parallel geöffnet haben.) Jeder Projektordner enthält eine Datei *package.bluej*, die mit BlueJ verknüpft ist und auf die Sie nur doppelklicken müssen, um das Projekt direkt zu öffnen.

Weitere Informationen über den Umgang mit BlueJ finden Sie im Tutorial zu BlueJ. Dieses Tutorial ist über den Eintrag BLUEJ-TUTORIAL im HILFE-Menü von BlueJ erreichbar.

A.3 Der Debugger in BlueJ

Informationen über die Benutzung des Debuggers finden Sie in Anhang F und im BlueJ-Tutorial. Das Tutorial ist über den Eintrag BLUEJ-TUTORIAL im HILFE-Menü von BlueJ erreichbar.

Lizenziert für Hochschule Luzern, 147.88.200.39 am 22.02.2023 um 10:51 Uhr

A.4 BlueJ konfigurieren

Sie können viele Einstellungen von BlueJ ändern, um BlueJ besser auf Ihre individuelle Situation abzustimmen. Einiges können Sie über den Dialog EINSTELLUNGEN in BlueJ verändern, aber viele weitere Konfigurationseinstellungen können geändert werden, indem Sie die Datei mit den BlueJ-Definitionen editieren. Der Pfad dieser Datei lautet *<bluej_home>/lib/bluej.defs*, wobei *<bluej_home>* den Ordner bezeichnet, in dem das BlueJ-System installiert ist.[1]

Details zur Konfiguration werden im „Tips archive" auf den BlueJ-Webseiten erläutert. Sie erreichen das Archiv über

http://www.bluej.org/help/archive.html

Im Folgenden sind die Punkte beschrieben, die am häufigsten angepasst werden. Viele weitere finden Sie, indem Sie sich die Datei *bluej.defs* genauer ansehen.

A.5 Auf deutsche Schnittstelle umstellen

Sie können BlueJ so konfigurieren, dass die Benutzungsschnittstelle in Deutsch angezeigt wird, indem Sie *bluej.defs* in einem beliebigen Editor editieren.

In der Datei finden sie unter anderem die folgenden Zeilen:

```
bluej.language=english
#bluej.language=afrikaans
#bluej.language=catalan
#bluej.language=chinese
#bluej.language=czech
#bluej.language=danish
#bluej.language=dutch
#bluej.language=french
#bluej.language=german
#bluej.language=greek
…
```

Ein Hash-Symbol (**#**) am Anfang der Zeile repräsentiert einen Kommentar. Die Zeile, die nicht mit einem Hash-Symbol beginnt, ist folglich die ausgewählte Einstellung (hier: **`bluej.language=english`**).

Sie können Deutsch als Benutzersprache auswählen, indem Sie diese Zeilen wie folgt ändern:

```
#bluej.language=english
#bluej.language=afrikaans
#bluej.language=catalan
#bluej.language=chinese
#bluej.language=czech
#bluej.language=danish
#bluej.language=dutch
```

1 Unter Mac OS ist die Datei *bluej.defs* im Anwendungspaket enthalten. Unter „Tips archive" finden Sie nähere Angaben, wo Sie die Datei finden können.

Lizenziert für Hochschule Luzern, 147.88.200.39 am 22.02.2023 um 10:51 Uhr

```
#bluej.language=french
bluej.language=german
#bluej.language=greek
```

Speichern Sie die Datei und starten Sie BlueJ.

A.6 Einbinden einer lokalen API-Dokumentation

Sie können eine lokale Kopie der Dokumentation der Java-Klassenbibliotheken (API) benutzen. Auf diese Weise erfolgt der Zugriff auf die Dokumentation schneller und sie müssen zum Lesen nicht online sein. Dazu laden Sie die Datei mit der Java-Dokumentation von *http://www.oracle.com/technetwork/java/javase/downloads/* herunter und entpacken sie an einem Ort, an dem Sie die Dokumentation ablegen wollen. Dies erzeugt einen Ordner *docs*.

Starten Sie dann einen Webbrowser und öffnen Sie mit diesem über die Funktion DATEI ÖFFNEN (oder einer ähnlichen Funktion) die Datei *docs/api/index.html*, die innerhalb des besagten Ordners liegt. Wenn die API-Ansicht korrekt im Browser angezeigt wird, kopieren Sie die URL (die Web-Adresse) aus dem Adressfeld des Browsers, öffnen BlueJ und darin den Dialog WERKZEUGE/EINSTELLUNGEN..., wählen den Reiter DIVERSES und fügen dort die URL in das Feld mit der Bezeichnung URL DER JDK-DOKUMENTATION ein. Über den Eintrag JAVA KLASSENBIBLIOTHEKEN im Menü HILFE sollten Sie nun Ihre lokale Kopie erreichen.

A.7 Vorlagen für neue Klassen ändern

Wenn Sie eine neue Klasse anlegen, wird der Quelltext der Klasse mit einem voreingestellten Text vorgegeben. Dieser Text wird einer Vorlage entnommen, die an Ihre Erfordernisse angepasst werden kann.

Die Vorlagen (*templates*) liegen in den Ordnern

<bluej_home>/lib/<Sprache>/templates/ und

<bluej_home>/lib/<Sprache>/templates/newclass/,

wobei *<bluej_home>* das Installationsverzeichnis von BlueJ ist und *<Sprache>* die aktuelle Spracheinstellung benennt (beispielsweise *german*).

Die Vorlagen sind reine Textdateien und können mit jedem Standardeditor bearbeitet werden.

Lizenziert für Hochschule Luzern, 147.88.200.39 am 22.02.2023 um 10:51 Uhr

Lizenziert für Hochschule Luzern, 147.88.200.39 am 22.02.2023 um 10:51 Uhr

ANHANG

B

Datentypen in Java

Java kennt zwei Arten von Typen: primitive Typen und Objekttypen.

Primitive Typen werden direkt in Variablen gespeichert und haben Wertsemantik (Werte werden bei einer Zuweisung an eine andere Variable kopiert). Primitive Typen haben nichts mit Klassen zu tun und verfügen nicht über Methoden.

Objekttypen werden als Referenzen auf Objekte gespeichert (nicht die Objekte selbst). Bei einer Zuweisung an eine andere Variable wird nur die Referenz kopiert, nicht das Objekt. (Sie erfahren mehr über die Referenzsemantik im Abschnitt B.3.)

B.1 Primitive Typen

Die folgende Tabelle führt alle primitiven Typen der Sprache Java auf:

Typname	Beschreibung	Beispielkonstanten		
Ganze Zahlen:				
`byte`	ganze Zahl in 8 Bit	`24`	`-2`	
`short`	ganze Zahl in 16 Bit	`137`	`-119`	
`int`	ganze Zahl in 32 Bit	`5409`	`-2003`	
`long`	ganze Zahl in 64 Bit	`423266353`	`55L`	
Gleitpunktzahlen:				
`float`	einfache Fließkommazahl (32 Bit)	`43.889F`		
`double`	doppelte Fließkommazahl (64 Bit)	`42.63`	`2.4e5`	
Andere Typen:				
`char`	einzelnes Zeichen (16 Bit)	`'m'`	`'?'`	`'\u00F6'`
`boolean`	ein boolescher Wert (wahr oder falsch)	`true`	`false`	

Lizenziert für Hochschule Luzern, 147.88.200.39 am 22.02.2023 um 10:51 Uhr